Industrial Equipment Health Monitoring & RUL Prediction SCADA

Comprehensive Technical Guide: SQLite Database, Data Preprocessing, Machine Learning Training & Inference

Target Hardware: RS-380 DC Motor Testbed	**Database:** SQLite 3 (equipment_health.db)	**ML Framework:** Scikit-Learn 1.3+ & SHAP
Dataset: 21,300 samples across 120 cycles	**Best Model:** Random Forest Regressor (R^2=0.8227)	**Status:** Production Ready & Deployed

1. Executive Summary & Architecture

This engineering report provides an in-depth breakdown of the **Intelligent Industrial Predictive Maintenance and Remaining Useful Life (RUL) Prognostics System** engineered for electromechanical motor testbeds. The platform continuously monitors high-speed physical telemetry acquired by an **ESP32 microcontroller** (sampling vibration, dual current channels, temperature, and individual lithium battery cell voltages at 10 Hz), stores real-time data into an embedded **SQLite time-series historian**, pre-processes signals using physics-informed boundary clamping and rolling statistical windows, executes an ensemble of **Random Forest, Gradient Boosting, and Isolation Forest** models, and renders an industrial SCADA Digital Twin web console.

Core Architecture Flow: Physical Sensors → ESP32 12-bit ADC1 & FreeRTOS → Dual Transport (USB Serial @ 115,200 baud & Wi-Fi UDP :8888) → Python SCADA Edge Server (serve.py) → SQLite Historian (equipment_health.db) → ML Feature Extraction & RUL Inference → Web SCADA Console (10 Hz live dashboard with real-time safety watchdog).

2. In-Depth SQLite Database Breakdown

Database Location: `data/equipment_health.db` (SQLite 3 binary database, ~6.8 MB on disk).
Schema Definition: `data/schema.sql`
Database Manager: `src/data/db.py (DatabaseManager)`

The database contains two high-throughput relational tables that capture both physical raw measurements and calculated machine intelligence outputs at 1 Hz intervals:

Table 1: raw_sensor_data (Physical Telemetry Stream)

Column Name	SQL Data Type	Physical Engineering Meaning
`id`	INTEGER PRIMARY KEY	Autoincrementing record identifier
`timestamp`	DATETIME	ISO-8601 acquisition timestamp (indexed for fast queries)
`device_id`	TEXT	Hardware node tag (defaults to 'RS380-ESP32-01')
`battery_voltage`	REAL	3S Lithium-Ion pack terminal voltage (11.1V - 12.6V)
`motor_voltage`	REAL	Voltage across L298N H-bridge motor terminals (V)
`total_current`	REAL	Total bus current from ACS712 #2 (Amperes)
`motor_current`	REAL	Motor armature load current from ACS712 #1 (Amperes)
`temperature`	REAL	Motor casing & stator temperature from DS18B20 (°C)
`vibration`	REAL	801S dynamic vibration acceleration magnitude (g)
`operating_mode`	TEXT	Operational tag: NORMAL, OVERLOAD, IMBALANCE, STALL

Table 2: health_features (Prognostic ML Predictions)

Column Name	SQL Data Type	Algorithmic Function
`id`	INTEGER PRIMARY KEY	Autoincrementing prediction record ID
`timestamp`	DATETIME	Timestamp corresponding to the telemetry snapshot
`health_index`	REAL	Overall machine health index ranging from 1.0 (new) to 0.0 (failed)
`rul_hours`	REAL	Predicted Remaining Useful Life in operating hours
`rul_ci_low`	REAL	95% Confidence Interval lower bound (hours)
`rul_ci_high`	REAL	95% Confidence Interval upper bound (hours)
`rms_deviation`	REAL	Normalized Mahalanobis-style deviation from baseline
`top_contributor`	TEXT	Sensor driving the greatest degradation per SHAP attribution
`top_contributor_pct`	REAL	Percentage share of degradation attributed to top sensor (%)
`status_label`	TEXT	Health category: HEALTHY, WARNING, or CRITICAL

Understanding Row Counts & How to View All Rows as a Database

Active Row Counts: The live database currently stores **26,860+ records** in `raw_sensor_data` and **26,860+ records** in `health_features`. The metric *302,154* seen on the System Info page represents lifetime telemetry samples logged across multiple long-duration experimental test runs.

3 Ways to View All Rows:

1. Web Audit Log Tab: Open the SCADA dashboard → click **Audit Log** in the sidebar → view live tabular records or click **Export CSV** to download all records into Microsoft Excel.

2. DB Browser for SQLite: Download free *DB Browser for SQLite* (sqlitebrowser.org) → open `data/equipment_health.db` → switch to **Browse Data** tab to view, search, and sort all rows.

3. Python Automation Query: Use the following script to export all rows:

```
import sqlite3, pandas as pd
conn = sqlite3.connect('data/equipment_health.db')
df_raw = pd.read_sql_query('SELECT * FROM raw_sensor_data ORDER BY id ASC;', conn)
df_health = pd.read_sql_query('SELECT * FROM health_features ORDER BY id ASC;', conn)
print(f'Total Raw Records: {len(df_raw)} | Health Predictions: {len(df_health)}')
df_raw.to_csv('exported_all_sensor_data.csv', index=False)
```

3. Data Preprocessing & Cleaning Pipeline

Real-world industrial data streams inevitably suffer from packet re-transmissions, electrical commutation spikes, ADC thermal drift, and transient sensor noise. The preprocessing pipeline applies multi-stage cleaning and mathematical transformations:

• **Deduplication of Sensor Records:** In serial/wireless telemetry, network jitter or UART buffer re-reads can send identical packets. Deduplication is enforced by tracking unique combinations of `(cycle_id, timestamp)`. In the browser dashboard, a hash set `new Set(telemetryLog.map(r => r.ts.getTime()))` rejects duplicate readings at the client intake.
• **Modality Clarification (Time-Series Telemetry vs. Images):** This system processes *continuous multi-variate numerical time-series sensor data* rather than camera images. However, high-frequency vibration acceleration waveforms from the 801S sensor are converted into statistical spectral representations (RMS energy and Kurtosis shock indicator) rather than storing redundant raw analog samples.
• **Physical Boundary Clamping (Outlier Removal):** Extreme inductive spikes from motor stall or brush arcing can generate false ADC readings. Measurements are strictly bounded to physical envelopes using `np.clip`:

Sensor Channel	Valid Operating Range	Outlier Clamping Rationale
Battery Voltage	`8.00 V – 13.50 V`	Prevents ADC divider floating errors; 3S Li-ion absolute limits
Motor Terminal Voltage	`5.00 V – 13.00 V`	L298N H-bridge saturation output limits
Motor Armature Current	`0.30 A – 6.50 A`	RS-380 no-load (0.35A) to dead-stall saturation (5.5A)
Total System Current	`0.50 A – 7.50 A`	Full DC bus power consumption boundary
Core Temperature	`20.0 °C – 95.0 °C`	Rejects DS18B20 OneWire bus disconnects (-127°C error value)
Vibration Acceleration	`0.05 g – 4.00 g`	ISO 10816-3 mechanical vibration limit floor and peak ceiling

• **Rolling Statistical Windowing (W=5 Samples, ~15 Seconds):** Instantaneous values fail to capture gradual degradation trends. For each sensor channel, a 5-sample rolling window calculates:
1. **Rolling Mean:** Bounded average filtering high-frequency noise.
2. **Rolling RMS:** Root Mean Square capturing effective continuous kinetic energy.
3. **Rolling Standard Deviation:** Quantifies mechanical vibration flutter and commutator sparking.
4. **Delta (Rate of Change):** First-order derivative indicating rapid thermal runaway or current surge.

• **Derived Multi-Physics Features:**
1. *Apparent Power (W):* `P = V_motor * I_motor` (reflects mechanical shaft loading).
2. *Voltage Sag Ratio:* `R_v = V_motor / V_battery` (detects battery internal resistance aging).
3. *Thermal Current Stress:* `S_thermal = Temperature * I_motor` (joule heating degradation on winding enamel).

• **StandardScaler Normalization:** All features are normalized to zero mean and unit variance. The scaler is fit strictly on the training partition and applied without refitting to validation and live telemetry.

4. The Machine Learning Dataset Breakdown

Dataset File: `data/processed/run_to_failure.csv` (14.8 MB, 21,300 rows × 38 columns).
Metadata Manifest: `data/processed/dataset_meta.json`
Data Generation Code: `src/data/generate_training_data.py`

The training dataset comprises **21,300 run-to-failure snapshots across 120 independent lifecycles**. The lifetime of each motor lifecycle is generated using a **Weibull Distribution (shape β=3.5, scale η=10.0 hours)**. A Weibull shape parameter β=3.5 strictly models mechanical wear-out fatigue (matching the NASA C-MAPSS and F.A.I.R. Brushed DC Motor benchmarks).

Degradation Mode	Share (%)	Cycles	Physics & Failure Signature
Healthy Aging	33.3%	40	Gradual carbon brush wear, light thermal drift (+11°C), commutator groove formation.
Rotor Imbalance	29.2%	35	Eccentric rotor / bearing play; vibration accelerates quadratically (t^2.1 up to 2.15g).
Mechanical Overload	27.5%	33	Bearing binding & stall friction; motor current surges up to 3.2A, strong battery sag.
Thermal Runaway	10.0%	12	Duty cycle abuse; exponential heat soak (+40°C), thermal insulation breakdown.

5. Machine Learning Model Training & Evaluation

Training Script: `src/models/train.py`
Serialized Model Pipelines: `models/rul_rf_pipeline.joblib` & `models/rul_gbr_pipeline.joblib`
Metrics Record: `models/metrics.json`

1. Data Leakage Prevention via GroupShuffleSplit:
Standard row-level random splitting causes severe artificial metric inflation because adjacent sensor readings from the same physical run share degradation context. To guarantee honest generalization, we split data strictly by `cycle_id` using **GroupShuffleSplit (80% train / 20% test)**:
• **Train Set:** 16,740 samples (96 independent motor lifecycles)
• **Test Set:** 4,560 samples (24 unseen motor lifecycles)

2. 5-Fold GroupKFold Cross-Validation:
During model tuning, a 5-fold `GroupKFold` evaluated model stability across groups. Validation RMSE across folds was **1.641 ± 0.098 hours** for Random Forest and **1.664 ± 0.105 hours** for Gradient Boosting.

Model Architecture	Hyperparameters	Test R^2	Test RMSE	Test MAE
Random Forest Regressor (Primary)	`n_est=150, max_depth=10, min_leaf=4`	**0.8227**	**1.3306 h**	**0.9890 h (59m)**
Gradient Boosting Regressor	`n_est=200, depth=5, lr=0.05, sub=0.8`	0.8194	1.3431 h	1.0057 h (60m)
Isolation Forest (Anomaly)	`n_est=50, contamination=0.05`	N/A (Unsup.)	ROC-AUC 0.94	Zero-Day Flag

3. Uncertainty Quantification (95% Confidence Intervals):
Single-point RUL estimates in industrial operations are insufficient for safety-critical decisions. By computing the standard deviation of individual decision tree predictions across the Random Forest ensemble (`sigma = std(tree_predictions)`), the system computes dynamic 95% Confidence Intervals: `[RUL_low = RUL - 1.96*sigma, RUL_high = RUL + 1.96*sigma]`.

6. Explainable AI with SHAP (TreeExplainer)

To ensure total transparency and comply with **ISO 13374 Condition Monitoring standards**, the system incorporates game-theoretic **SHAP (SHapley Additive exPlanations)**. Using `shap.TreeExplainer`, the model quantifies the exact contribution percentage of each sensor toward equipment degradation:

Physical Sensor Channel	SHAP Contribution Share (%)	Observed Physical Degradation Mechanism
Motor Core Temperature	**46.3%**	Stator coil insulation degradation, bearing friction heating, and thermal fatigue.
Vibration Acceleration	**22.2%**	Shaft eccentricity, commutator surface pitting, and mechanical bearing wear.
Battery Pack Voltage & Sag	**10.0%**	Internal battery cell resistance increase under load and undervoltage sag.
Motor Armature Current	**8.5%**	Commutator arcing, increased brush resistance, and mechanical load bind.
Total System Bus Current	**7.6%**	Overall electrical efficiency drops and H-bridge switching losses.
Motor Terminal Voltage	**5.4%**	Driver MOSFET voltage drop and supply rail fluctuations.

7. Live SCADA Real-Time Inference & Watchdog Interlock

Inference Module: `src/models/predict.py (RULPredictor)`
Server Core: `dashboard/serve.py`
Configuration File: `config/calibration.json`

During live operations, the Python server executes a continuous 10 Hz acquisition loop. Every incoming packet undergoes immediate ML feature transformation and safety limit evaluation:
1. **Real-Time Inference:** Features are scaled and fed into `rul_rf_pipeline` and `anomaly_iso_forest` in under 12 milliseconds.
2. **Dynamic Calibration Studio:** In the Web Calibration page, users can adjust safety trip thresholds (Current max 4.5A, Vibration max 2.5g, Temperature max 85°C, BMS cell min 2.80V) with two-way sync.
3. **Automated Safety Watchdog Interlock:** If any sensor exceeds the configured trip limit in live telemetry, the watchdog triggers an automated emergency stop (`STOP / ESTOP`) over the USB/UDP link within <100ms and engages the hardware trip interlock.

8. Complete Setup & Verification Instructions

Option A: 1-Click Launch (Windows):
Double-click `START_SYSTEM.bat`. The script checks Python, installs dependencies from `requirements.txt`, connects to the ESP32 on COM5 / UDP 8888, launches the server on port 8000, and opens your web browser.
Option B: Manual Terminal Execution:
1. Install dependencies: `pip install -r requirements.txt`
2. Re-train models (optional): `python src/models/train.py`
3. Start SCADA server: `python dashboard/serve.py 8000`
4. Access dashboard: **http://localhost:8000**